Bounded by Specification: Calibrated Per-Phase Deadlines for Independent Runtime Monitoring of Robot Skills

[TODO: authors, affiliations, funding, RA-L footnotes]

***Abstract*—A runtime monitor for a robot skill is usually handed the skill's goal as a temporal-logic formula. That formula is a guarantee property: it has no bad prefix, so a monitor can confirm it and never refute it, and a stalled robot leaves it inconclusive forever. In practice the gap is filled by the controller timing itself out, which fails exactly when the controller is wrong about its own state. We present a skill specification that pairs Linear Temporal Logic (LTL) invariants with a ladder of *phases*, each carrying a precondition, an invariant, a progress condition with a violation budget, an exit condition and a deadline. The monitor that runs it is independent of the controller it judges, reaches a verdict within a horizon fixed by the specification, and names the stage and guard that failed. The deadlines are not hand-tuned: they are calibrated as conformal quantiles of phase durations in successful runs, so the false-alarm level is stated rather than hoped for. We evaluate the monitor as an intervention rule — detection delay at a fixed false-alarm level, break-even false-alarm rate and execution saved — on teleoperated humanoid pick-and-place and on humanoid navigation, against a flat specification, a mission-level deadline and per-stage bounded LTL. [TODO: headline result]**

***Index Terms*—Formal methods in robotics and automation, failure detection and recovery, runtime verification, humanoid robots.**

I. Introduction

A HUMANOID executing a waypoint mission wedges itself against a lip its traversability estimate called flat. The legs keep cycling, the planner keeps reporting that it is following a path, and the goal is never reached. What should a runtime monitor say?

The skill's requirement, in LTL, is something like

$$\mathbf{F}\big(\mathit{armed} \wedge \mathbf{F}(\mathit{planning} \wedge \mathbf{F}(\mathit{walking} \wedge \mathbf{F}\mathit{finished}))\big), \quad (1)$$

which is the goal formula our own platform shipped with. A monitor for (1) reports INCONCLUSIVE on this trace, and on every later tick. It is not late; it is correct. The formula is a guarantee property: at any finite point *finished* could still occur, so there is no bad prefix and nothing to refute [1], [2]. The formula that says what a skill is *for* is the one that can never report that the skill failed.

Practice fills the gap with timeouts, and they live in the controller: a navigation stack's progress checker, a behaviour tree's timeout decorator, an action server's deadline. These work until the controller is wrong about itself, which is the case above — any check that reads the planner's status sees `following`.

We move the bound out of the controller and into the specification. A skill is described as cross-cutting LTL invariants plus a ladder of **phases**; each phase declares what must hold, what counts as progress and how long it may take. Figure 1 shows one mission both ways. The resulting monitor reads the robot rather than the planner, cannot wait forever, and reports `phase:tracking:timeout` rather than a failed conjunction.

A bound must come from somewhere, and a hand-picked one invites the charge that it was tuned to the failures. We calibrate each phase's deadline from successful runs as a conformal quantile of its duration, which fixes the false-alarm level in advance. And since an alarm is only worth what the response to it achieves [3], we evaluate the monitor as an intervention rule, not as a classifier.

Contributions.

1) *Analysis.* A skill's goal formula is a guarantee property and is never refuted online under LTL_3; neither a finite-trace nor a four-valued semantics repairs this. A bound per stage does, with a proved horizon on time to verdict (Section IV).
2) *Design.* A two-layer specification — LTL automata for cross-cutting invariants, a phase ladder for bounded, localised and categorised progress obligations — whose bounds are calibrated from nominal runs at a stated false-alarm level.
3) *Independence.* The monitor is an artifact separate from the controller and grounded in the robot's own signals; we exhibit the case in which the controller's own progress check is silent.
4) *Evaluation as an intervention rule* on two skill families on one humanoid, against three baselines. **[TODO: result]**

II. Background: LTL, and Why LTL

LTL in brief. LTL [4] extends propositional logic over atomic propositions AP with operators over discrete time: $\mathbf{X}\varphi$ (next), $\mathbf{G}\varphi$ (always), $\mathbf{F}\varphi$ (eventually) and until. A model is an infinite word over 2^{AP}; here one position is one monitor tick. A formula is compiled into an automaton whose state the monitor tracks as observations arrive [5]; we use Spot [6]. Two property classes matter [1]. A *safety* property, such as $\mathbf{G}\neg\mathit{collision_risk}$, is violated by a finite bad prefix. A *guarantee* property, such as (1), has none: every finite trace has an extension that satisfies it.

Monitoring a finite observation. LTL_3 [2] maps a finite prefix to $\top$ if every infinite continuation satisfies the formula,

Flat LTL	Phase ladder
$\mathbf{F}(armed \wedge \mathbf{F}(planning \wedge \mathbf{F}(walking \wedge \mathbf{F}finished)))$	PLANNING: deadline 30, budget 5
Stalled trace: INCONCLUSIVE, forever. No diagnosis.	TRACKING: deadline 120, budget 5
	CONFIRM: deadline 10
Stalled trace: `phase:tracking:timeout` → REPLAN.	

Fig. 1. One navigation mission, two encodings, from the same specification file (bounds in ticks). **[TODO: replace with a drawn figure and an annotated stall trace.]**

$\perp$ if none does, and ? otherwise. It is the semantics of observing an open-ended run, which is the monitor's situation at every tick, and we adopt it for the verdict that matters: our VIOLATED is LTL_3's $\perp$, reported when the automaton enters a non-accepting sink, and it is permanent. (Our ACCEPTED status only reports an accepting current state and is not LTL_3's $\top$; nothing acts on it.) Two alternatives do not help online. LTL_f [7] would declare $\mathbf{F}$ *finished* false on a finite trace, but only at the last position, and it cannot say when the trace ends — something outside the logic must, and in our system that is a deadline. RV-LTL [8] refines ? into "presumably true" and "presumably false", but $\mathbf{F}$ *finished* is presumably false on every prefix of a *successful* run until its last tick, so the value does not separate stalled from healthy and is not monotone.

Why LTL as the description language. We need three things at once. *A declared property rather than an executable.* Behaviour trees and state machines are how robot behaviour is structured, and runtime verification has been built around them [9], [10], [11]; but the property a tree enforces is implicit in its traversal, and in practice the tree is the controller, so a monitor derived from it shares the controller's view of the world. *A verdict that cannot be retracted.* A violated safety formula sits in a sink state, so a supervisor can latch on it; a learned or language-model judgement can change on the next frame. *A property class one can reason about*, which is what exposes the unrefutable goal in the first place. Signal Temporal Logic [12], [13] is the serious alternative and does offer bounded operators; we keep the temporal layer Boolean because a robustness value says how badly rather than which obligation, and we push thresholds into proposition grounding. Nothing in the phase structure precludes STL guards.

III. Related Work

Stage structure in monitoring. That execution monitoring needs a notion of stage is old [14], [15] and now common. It is *learned* from labelled episodes: RoboMonitor predicts phase, failure and completion with a vision–language model and must suppress spurious phase switching [16]; ConditionNET learns preconditions and effects and compares expected with predicted phase [17]. It is *consumed*: AHA and Guardian verify a given sub-task from keyframes [18], [19], and REFLECT explains a failure afterwards [20]. It is *generated*: DoReMi has a language model emit per-skill constraints that a vision model checks during execution [21], and Code-as-Monitor compiles during-execution and on-completion constraints into code [22], a split close to our invariant and exit condition. And it is used to make *planning* tractable through hierarchical LTL [23], whose sub-formulas remain LTL and so inherit the property class of their parent. In the systems we examined the per-stage obligations are states to maintain; where a time bound appears it is a single task-level timeout.

Controller-side progress checks. Horizons on progress do exist, inside controllers. Nav2's progress checker fails a controller that has not moved a set distance within a set time [24]; behaviour trees have timeout decorators [9]; a steps-since-progress counter is a standard monitoring baseline [3]. We do not claim the mechanism. We claim its placement — outside the component it judges, declared per stage in a specification, producing a categorised verdict — and its calibration. The risk of the alternative is documented outside robotics too: language-model agents asked to report their own progress are unreliable mid-task, and such reports should be checked against independent state [25].

Learned failure detection and its evaluation. A large literature detects failures of learned policies from their own statistics or from visual judgement [26], [27]. Two results from it shape our evaluation. FAIL-Detect calibrates its threshold by conformal prediction on successful runs only, obtaining a false-positive guarantee without failure data [28]; we apply the same idea to phase deadlines. RESCUE shows that detection and timing metrics admit monitors that are harmful once false alarms and intervention cost are counted, and gives a break-even false-alarm rate computable from execution logs [3]; we report it. Granularity matters in neighbouring fields as well: stage-wise evaluation of long-horizon agents discriminates better than whole-trajectory scoring [29].

Specifications from language. Translating instructions into temporal logic is well studied [30], [31], [32], [33]. Our specifications can be generated from a skill description and checked against a declared sensor schema; we do not evaluate that here.

Position. Stage structure is prior art, and so is the timeout. What we add is a specification in which each stage carries a calibrated bound, a monitor for it that is independent of the controller, the property-class argument for why the bound is necessary rather than convenient, and an evaluation of the result as an intervention rule.

IV. A Phase-Structured Specification

A. The gap

A skill specification contains two kinds of formula. Cross-cutting invariants such as $\mathbf{G}\neg collision_risk$ and $\mathbf{G}\, upright$ are safety properties and are refutable. The goal formula (1) is a guarantee property, so its LTL_3 verdict is $\top$ or ?, never $\perp$. A robot that is wedged, circling or standing still produces an unbounded run of inconclusive verdicts. A *terminal failure condition* does not close the gap if it is written over the controller's status, as ours was.

B. Phases

A phase is a named stage with five guards and two budgets: an *enter condition*; a *precondition*, checked once on entry; an *invariant*, checked every tick; a *progress condition* with a *violation budget* m, exhausted by m consecutive failures; an *exit condition*, which advances the ladder; and a *deadline* k in ticks. Each breach is a named fault, `phase:`⟨`name`⟩`:`⟨`kind`⟩, with a category.

Bounding an obligation is what changes its class: "this phase is eventually left" has no bad prefix, while "this phase is left within k ticks" is decided in $k+1$ positions. That is a fact about the bound. For the ladder it yields a horizon on time to verdict.

Proposition 1: Let a ladder have phases $1 \dots n$ visited in order, each declaring a finite deadline k_i. Within $\sum_{i=1}^{n}(k_i+1)$ ticks of entering the first phase, the monitor has completed the ladder or raised a fault.

Proof. The ladder occupies one phase at a time. A phase's tick counter is reset on entry and incremented on every tick the phase remains active. On any tick at which it has reached k_i the machine raises a terminal timeout fault before consulting the progress or exit condition. So a phase is occupied for at most $k_i + 1$ ticks; phases are left only forwards and never re-entered. ∎

The proposition is elementary, and we do not offer it as a result: it says only that non-completion always yields a verdict, not that the verdict is timely or deserved. Its value is what it rests on. The timeout path counts ticks and evaluates no expression, so it holds whatever the sensors or the controller report. The progress budget is the opposite: it is what brings typical delay far below the bound, and it is only as independent as the proposition it reads. Whether the verdict is timely, and what it costs, is empirical (Section VI).

What the phases add beyond the bound: a progress budget needs a stage to be scoped to; an overrun is caught in the stage where it occurs; and the stage index answers the operator's first question and gives one place to grade a fault into an intervention. Per-stage *bounded* LTL, $\mathbf{G}(\mathit{armed} \wedge \neg \mathit{planning} \rightarrow \mathbf{F}_{\leq k}\, \mathit{planning})$, expresses the deadlines without a phase machine, and we do not claim otherwise; it gives no stage index, its automata grow with k, and scoping a budget still needs a stage indicator. We treat it as a baseline, not an argument.

C. Calibrating the bounds

A deadline's false-alarm rate is the probability that a healthy run overstays it, so choosing k_i is choosing a quantile of the phase's nominal duration. We make that explicit. From N successful runs we record each phase's duration $d_i^{(1)}, \dots, d_i^{(N)}$ and set

$$k_i \;=\; \text{the } \lceil (N+1)(1-\alpha_i) \rceil\text{-th smallest of } d_i^{(1)}, \dots, d_i^{(N)},$$

the split-conformal quantile used for failure-detection thresholds in [28]. If a new healthy run is exchangeable with the calibration runs, the phase overstays k_i with probability at most α_i; taking $\alpha_i = \alpha/n$ bounds the episode-level false-alarm probability from deadlines by α. The progress budget m is calibrated the same way from the longest run of consecutive progress violations in each successful run. Two caveats are intrinsic. The guarantee is marginal and needs $N \geq (1-\alpha_i)/\alpha_i$ runs to be non-trivial — below that k_i is simply the longest duration seen. And exchangeability fails when the operator, the scene or the robot changes, which is why the level is checked on held-out runs rather than assumed.

D. What it does not fix

The bounds and grounding thresholds are calibration, not logic: a wrong one yields a verified wrong answer. The ladder is a total order and does not express concurrent stages. It cannot flicker between stages, but a one-tick glitch in an exit condition advances it irreversibly. Finally, a proposition that cannot be evaluated is handled inconsistently today: a phase guard falls back to a pass, while the automata read a missing proposition as false, so a safety fault can be hidden or fabricated depending on how its proposition is named. A three-valued treatment that freezes the monitor and raises a data-loss fault is designed and not deployed.

V. The Monitor

Artifacts. A *sensor schema* declares the keys a specification may mention. An *adapter descriptor*, one per embodiment, maps topics and fields to those keys and declares how samples within a tick fold to one value. A *skill specification* holds the propositions with their grounding rules, the LTL invariants with fault categories, and the phase ladder. The monitor reads a specification and an observation stream and never the descriptor, so nothing in it names a robot; the same engine runs both skills in Section VI.

One tick. The monitor is driven by a clock, not by message arrival, so a bound in ticks means the same thing live and on replay. On each pulse: (i) each key's samples fold to one value; (ii) the propositions the monitor currently needs are grounded — a rule of the form "True when ⟨expression⟩" is evaluated directly, and a proposition that is not reducible to a threshold is sent to a model on a worker thread; (iii) each automaton takes one step; (iv) the active phase's guards are consulted in the order invariant, deadline, progress, exit; (v) one verdict is published with the phase, each formula's status, the truth of each guard, the violation counters and any fault.

From fault to intervention. A monitor alone changes no outcome. A fault carries a category: timeout and progress are fixed, and each phase's invariant and precondition declare theirs. Category, imminence and confidence select a rung of a six-step ladder — continue, warn, slow, replan, halt, abort — graded in one place. A confirmed safety fault halts or aborts; a confirmed timeout or stall requests a replan; a fault resting on low-confidence sensing is reduced to a warning. A supervisor overrides the base with zero velocity while a halting fault is active and does nothing until enabled, which provides a detection-only arm.

VI. Evaluation

A. Data

Teleoperated pick-and-place (main). **[TODO: N]** episodes of a Unitree G1 picking and placing a bottle under

teleoperation, with egocentric video and joint values per frame, labelled *successful*, *failed* or *almost failed*. The skill is naturally staged — reach, grasp, lift, transport, place — and is the task family most of the related work uses. Propositions are grounded from joint values through forward kinematics (hand position, gripper closure, hand activity) and, for object state, from video. Phase boundaries in successful episodes and the onset frame in failed ones are annotated from video alone, so no label is derived from a signal the monitor reads.

Humanoid navigation (supporting). Waypoint missions on the same robot and in MuJoCo [34] simulation, with stalls induced at known times. This is the setting in which a controller-side check exists and can be silent.

Why these and not a public benchmark. We know of no public set with multi-stage skills, stage annotations and controller-status signals together. Manipulation failure sets built by perturbing demonstrations [18], [19], [20] lack the last; the closest navigation set, ForEnt, records quadruped entrapments in single-stage traversal [35] and is a natural next test of the progress budget. The recordings here have a limit we state now: nothing in them can be intervened on, so we measure what a monitor would have saved, not task success.

B. Arms and metrics

All arms replay identical observation streams and share propositions and invariants. **B1**: flat goal formula. **B2**: B1 plus one mission deadline, calibrated like ours. **B3**: per-stage bounded LTL without a phase machine. **C**: the phase ladder. For navigation, **B4**: the controller's own progress check.

We report, per failure class: *false-alarm rate* on held-out successful episodes, against the level α fixed before any failed episode was examined; *detection delay* from annotated onset, as median and worst case over detected episodes, with the detection rate given separately because delay is censored when nothing is detected; *localisation*, whether the fault names the annotated stage; *execution saved*, frames from fault to episode end; and the *break-even false-alarm rate* of [3], the level above which acting on the monitor costs more than ignoring it. For almost-failed episodes we report the highest intervention rung reached: the intended behaviour is a warning that clears, not a terminal fault. We do not report AUROC, which scores frames and is blind to when an alarm arrives.

Detection rate is not a finding for B1 or C. B1 detects no progress failure by the argument of Section IV, and C raises a fault on every non-completing episode by Proposition 1. Both rates are reported as a check on the implementation. The empirical questions are whether the calibrated false-alarm level holds, how far below the bound the delay falls, and whether C improves on B2 and B3.

C. Results

[TODO: E1, pick-and-place: Table I; does the held-out false-alarm rate respect α; delay of C against B2 and B3; localisation accuracy; behaviour on almost-failed episodes; one figure of delay against false-alarm rate as α varies.]

[TODO: E2, navigation: the wedged-while-`following` case, in which B4 and any planner-status proposition are silent and only bounds grounded in the robot's own motion fire; delay per arm.]

[TODO: E3, cost: per-tick monitor time, split into automaton steps, guard evaluation and proposition grounding.]

TABLE I
PICK-AND-PLACE, α = [TODO:]. DELAY IN FRAMES.

arm	false alarm	near-miss fault	det. rate	median delay	b
B1 flat	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TO
B2 mission deadline	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TO
B3 bounded LTL	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TO
C phase ladder	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TO

VII. Limitations and Conclusion

Limitations. The main data are recordings, so the effect of intervention on task success is not measured; break-even false-alarm rate and execution saved are the proxies. Teleoperated runs have no controller status, so independence is exercised only in navigation. The calibration guarantee assumes exchangeable runs and is checked, not proved, for new operators and scenes. Our shipped navigation specification still grounded progress in the planner's status, which is the very coupling this paper argues against and is why only its deadline catches the motivating case; results use progress derived from the robot's own motion. Missing propositions are not yet handled uniformly (Section IV). One robot, two skills, sequential phases only.

Conclusion. The formula that states a skill's goal can be confirmed and never refuted, so the commonest failure of a robot skill — making no progress — is invisible to a flat temporal specification, and is left to the controller to notice about itself. Declaring a calibrated bound per phase in the specification puts that judgement in an independent monitor, fixes a horizon on its verdict, and attaches a stage, a guard and a response to the fault. LTL is kept alongside for the invariants and for the permanent verdict a supervisor can latch. **[TODO: one sentence of result.]**

References

[1] Z. Manna and A. Pnueli, "A hierarchy of temporal properties," in *Proc. 9th ACM Symposium on Principles of Distributed Computing (PODC)*, 1990, pp. 377–410.

[2] A. Bauer, M. Leucker, and C. Schallhart, "Runtime verification for LTL and TLTL," *ACM Transactions on Software Engineering and Methodology*, vol. 20, no. 4, 2011.

[3] D. Ahmedt-Aristizabal, A. Khamis, M. A. Armin, and L. Petersson, "RESCUE: Is this alarm worth acting on? counterfactual evaluation of VLA runtime monitors," *arXiv preprint arXiv:2610.13516*, 2026.

[4] A. Pnueli, "The temporal logic of programs," in *Proc. 18th Annual Symposium on Foundations of Computer Science (FOCS)*, 1977, pp. 46–57.

[5] M. Y. Vardi and P. Wolper, "An automata-theoretic approach to automatic program verification," in *Proc. 1st IEEE Symposium on Logic in Computer Science (LICS)*, 1986, pp. 332–344.

[6] A. Duret-Lutz, A. Lewkowicz, A. Fauchille, T. Michaud, É. Renault, and L. Xu, "Spot 2.0 — a framework for LTL and ω-automata manipulation," in *Proc. 14th International Symposium on Automated Technology for Verification and Analysis (ATVA)*, ser. LNCS, vol. 9938. Springer, 2016, pp. 122–129.

[7] G. D. Giacomo and M. Y. Vardi, "Linear temporal logic and linear dynamic logic on finite traces," in *Proc. 23rd International Joint Conference on Artificial Intelligence (IJCAI)*, 2013, pp. 854–860.

[8] A. Bauer, M. Leucker, and C. Schallhart, "Comparing LTL semantics for runtime verification," *Journal of Logic and Computation*, vol. 20, no. 3, pp. 651–674, 2010.

[9] M. Colledanchise and P. Ögren, *Behavior Trees in Robotics and AI: An Introduction*. CRC Press, 2018.

[10] M. Colledanchise, G. Cicala, D. E. Domenichelli, L. Natale, and A. Tacchella, "Formalizing the execution context of behavior trees for runtime verification of deliberative policies," in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2021.

[11] F. Ahmad, H. Ismail, J. Styrud, M. Stenmark, and V. Krueger, "A unified framework for real-time failure handling in robotics using vision-language models, reactive planner and behavior trees," in *Proc. IEEE 21st International Conference on Automation Science and Engineering (CASE)*, 2025, pp. 887–894.

[12] O. Maler and D. Nickovic, "Monitoring temporal properties of continuous signals," in *Formal Techniques, Modelling and Analysis of Timed and Fault-Tolerant Systems (FORMATS/FTRTFT)*, ser. LNCS, vol. 3253. Springer, 2004, pp. 152–166.

[13] G. E. Fainekos and G. J. Pappas, "Robustness of temporal logic specifications for continuous-time signals," *Theoretical Computer Science*, vol. 410, no. 42, pp. 4262–4291, 2009.

[14] O. Pettersson, "Execution monitoring in robotics: A survey," *Robotics and Autonomous Systems*, vol. 53, no. 2, pp. 73–88, 2005.

[15] G. D. Giacomo, R. Reiter, and M. Soutchanski, "Execution monitoring of high-level robot programs," in *Proc. International Conference on Principles of Knowledge Representation and Reasoning (KR)*, 1998.

[16] A. Ajith, G. Narayanan, K. Coelho, T. Zhao, Y. Shahapurkar, B. Zhu, M. Erdogan, T. Krubasik, C. Chamzas, and E. Solowjow, "RoboMonitor: Label-efficient runtime monitoring of robot task execution via predictive representation learning," *arXiv preprint arXiv:2609.30715*, 2026.

[17] D. Sliwowski and D. Lee, "ConditionNET: Learning preconditions and effects for execution monitoring," *IEEE Robotics and Automation Letters*, vol. 10, no. 2, pp. 1337–1344, 2025.

[18] J. Duan, W. Pumacay, N. Kumar, Y. R. Wang, S. Tian, W. Yuan, R. Krishna, D. Fox, A. Mandlekar, and Y. Guo, "AHA: A vision-language-model for detecting and reasoning over failures in robotic manipulation," in *Proc. International Conference on Learning Representations (ICLR)*, 2025.

[19] P. Pacaud, R. Garcia, S. Chen, and C. Schmid, "Guardian: Detecting robotic manipulation failures with vision-language models," *arXiv preprint arXiv:2512.01946*, 2025.

[20] Z. Liu, A. Bahety, and S. Song, "REFLECT: Summarizing robot experiences for failure explanation and correction," in *Proc. Conference on Robot Learning (CoRL)*, ser. PMLR, vol. 229, 2023, pp. 3468–3484.

[21] Y. Guo, Y.-J. Wang, L. Zha, and J. Chen, "DoReMi: Grounding language model by detecting and recovering from plan-execution misalignment," in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024.

[22] E. Zhou, Q. Su, C. Chi, Z. Zhang, Z. Wang, T. Huang, L. Sheng, and H. Wang, "Code-as-monitor: Constraint-aware visual programming for reactive and proactive robotic failure detection," *arXiv preprint arXiv:2412.04455*, 2024.

[23] S. Xu, X. Luo, Y. Huang, L. Leng, R. Liu, and C. Liu, "Nl2Hltl2Plan: Scaling up natural language understanding for multi-robots through hierarchical temporal logic task representation," *arXiv preprint arXiv:2408.08188*, 2024.

[24] S. Macenski, F. Martín, R. White, and J. G. Clavero, "The marathon 2: A navigation system," in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020.

[25] B. Wang, Y. Wang, and Y. Wu, "The unreliable progress bar: Can LLM agents reliably report task progress throughout execution?" *arXiv preprint arXiv:2609.08589*, 2026.

[26] C. Agia, R. Sinha, J. Yang, Z. ang Cao, R. Antonova, M. Pavone, and J. Bohg, "Unpacking failure modes of generative policies: Runtime monitoring of consistency and progress," in *Proc. Conference on Robot Learning (CoRL)*, ser. PMLR, vol. 270, 2024, pp. 689–723.

[27] A. V. Periasami, X. Luo, H. Li, and X. Cheng, "FRAMES: Failure recovery and monitoring of embodied skills for humanoid loco-manipulation," *arXiv preprint arXiv:2609.22538*, 2026.

[28] C. Xu *et al.*, "Can we detect failures without failure data? uncertainty-aware runtime failure detection for imitation learning policies," in *Proc. Robotics: Science and Systems (RSS)*, 2025.

[29] Z. Shi, W. Zhang, X. Jiang, X. Wu, C. Yang, C. Xu, J. Guo, and Y. Wang, "DynSTEER: Dynamic stage-wise trajectory evaluation and execution-time review for agents," *arXiv preprint arXiv:2609.14637*, 2026.

[30] J. X. Liu *et al.*, "Grounding complex natural language commands for temporal tasks in unseen environments," in *Proc. Conference on Robot Learning (CoRL)*, ser. PMLR, vol. 229, 2023.

[31] Y. Chen, R. Gandhi, Y. Zhang, and C. Fan, "NL2TL: Transforming natural languages to temporal logics using large language models," *arXiv preprint arXiv:2305.07766*, 2023.

[32] F. Fuggitti and T. Chakraborti, "NL2LTL — a python package for converting natural language instructions to linear temporal logic formulas," in *Proc. AAAI Conference on Artificial Intelligence (Demonstration Track)*, 2023.

[33] D. S. Sundarsingh, J. Wang, J. V. Deshmukh, and Y. Kantaros, "ConformalNL2LTL: Translating natural language instructions into temporal logic formulas with conformal correctness guarantees," *arXiv preprint arXiv:2504.21022*, 2025.

[34] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012, pp. 5026–5033.

[35] N. Kirdwichai and D. Tarapore, "ForEnt: A multi-modal dataset for characterizing quadruped robot entrapments in forest environments," *arXiv preprint arXiv:2606.19675*, 2026.